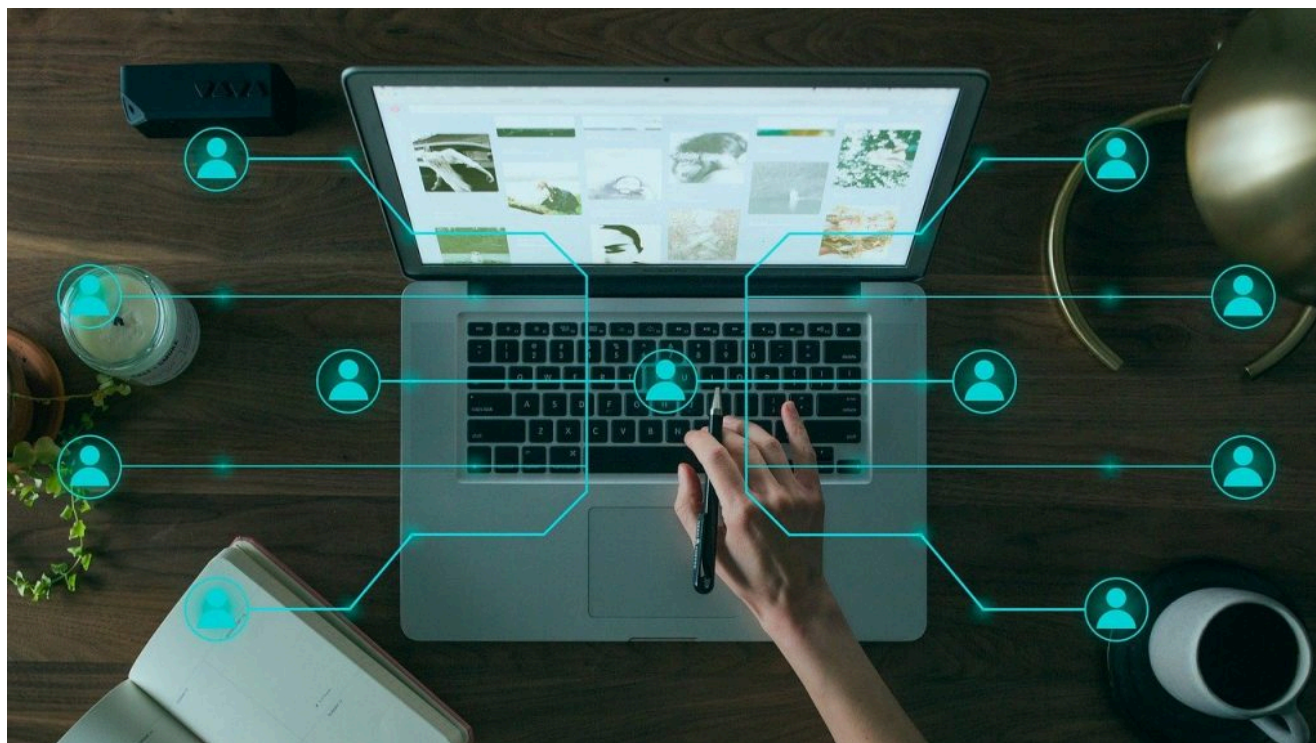


ОБЛАЧНЫЕ СЕРВИСЫ / СЕТЕВЫЕ ТЕХНОЛОГИИ

Эволюция в сторону сервис-ориентированной сети

Внедрение сетевых сервисов позволяет компаниям идти в ногу со скоростью разработки современных приложений без ущерба для управления и безопасности.

31 марта 2022 года, 7:04 [Питер МакКэррон](#)



Изображение для заставки предоставлено [Pixabay](#)

[HashiCorp](#) спонсировала этот пост.

На первый взгляд кажется очевидным, что эпоха облачных технологий коренным образом изменила **наши представления о сетевых технологиях**. Теперь мы работаем за пределами четко обозначенных периметров, а сети могут охватывать несколько центров обработки данных или облаков. Но так ли сильно изменились сетевые технологии по сравнению с теми временами, когда все размещалось в локальных центрах обработки данных?

В конце концов, суть по-прежнему заключается в обеспечении стабильного подключения и соблюдении политик безопасности. Так почему же, когда дело

Чтобы лучше понять эволюцию **современных сетевых технологий**, важно сделать шаг назад и определить основные рабочие процессы, которые привели к этим изменениям, в том числе:

- Поиск сервисов
- Защита сетей
- Автоматизация сетевых задач
- Контроль доступа

В этой статье мы рассмотрим каждый из этих рабочих процессов и поговорим о том, как их объединить для создания современного сетевого решения на основе сервисов. Поскольку я работаю в HashiCorp, я буду использовать **HashiCorp Consul** — инструмент с открытым исходным кодом — в качестве контекста, примеров и ссылок. Конечно, существуют и другие решения, которые можно использовать для достижения аналогичных результатов, но я лучше всего знаком с Consul.

Service Discovery

Можно с уверенностью сказать, что идентификация приложений или сервисов — не новая концепция. Специалисты по сетевым технологиям всегда хотели знать, какие приложения и сервисы работают в той или иной сети, где они расположены и доступны ли они. Новым стало резкое увеличение частоты изменений в этих средах. Переход к моделям непрерывной поставки и микросервисам привел к возникновению узкого места в виде ручного отслеживания. Использование электронных таблиц или **базы данных управления конфигурацией (CMDB)** для отслеживания IP-адресов не подходит для облачных сред. Что же оставалось делать сетевым операторам?



Инфраструктура способствует инновациям. HashiCorp предлагает согласованные рабочие процессы для подготовки, защиты, подключения и эксплуатации любой инфраструктуры для любого приложения.

Узнать больше →

Boundary 1.0 releases RDP session recording and improved management

25 June 2026

Scaling without friction: Aliases at project scope in Boundary

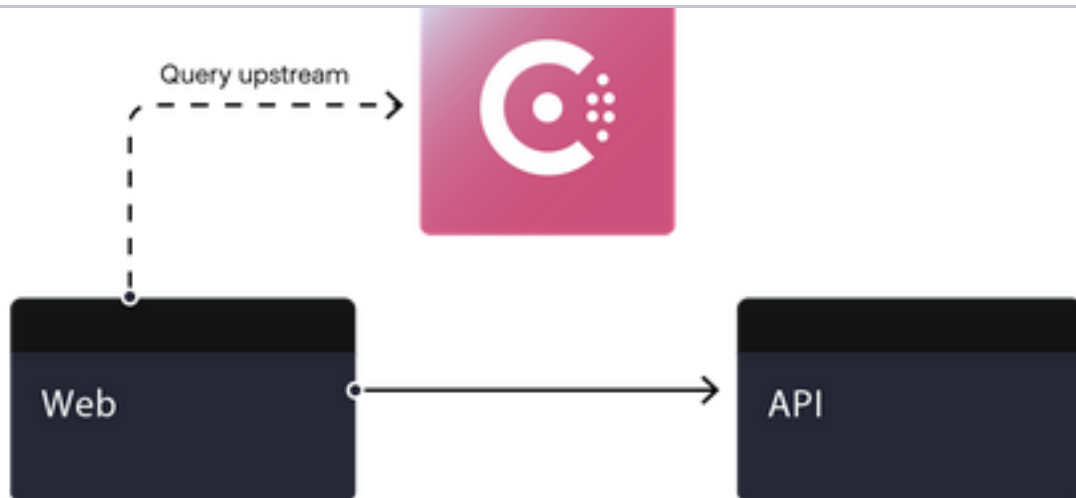
25 June 2026

Enter the concept of service discovery. Service discovery is just automating the process of identifying what is running on a network, knowing where it's located and if it's reachable in real time.

The need for this type of automation is what led to the creation of service discovery tools like Consul or Zookeeper. When **Consul was first introduced** in 2014, its core function was to be a dynamic service registry — essentially a catalog for tracking applications. With this kind of registry, users could package automation scripts along with application deployments to automatically register applications in these catalogs, creating a single source of truth for application information.

TRENDING STORIES

1. [HTTP/3 Is Now a Standard: Why Use It and How to Get Started](#)
2. [Linux: Display and Manage IP Address Settings](#)
3. [How To Read a Traceroute for Network Troubleshooting](#)
4. [From system of record to system of control: How NetBox Labs is making network engineers "masters of intent."](#)
5. [How Bolt Scaled Remote Work with Tailscale's Zero Trust Mesh VPN](#)

[Zoom](#)

As application delivery evolved, orchestrators such as Kubernetes, Mesos and Docker Swarm integrated discovery functionality to reduce the need for those manual scripts. And while that's great, what does it mean for the evolution of networking? A lot, actually.

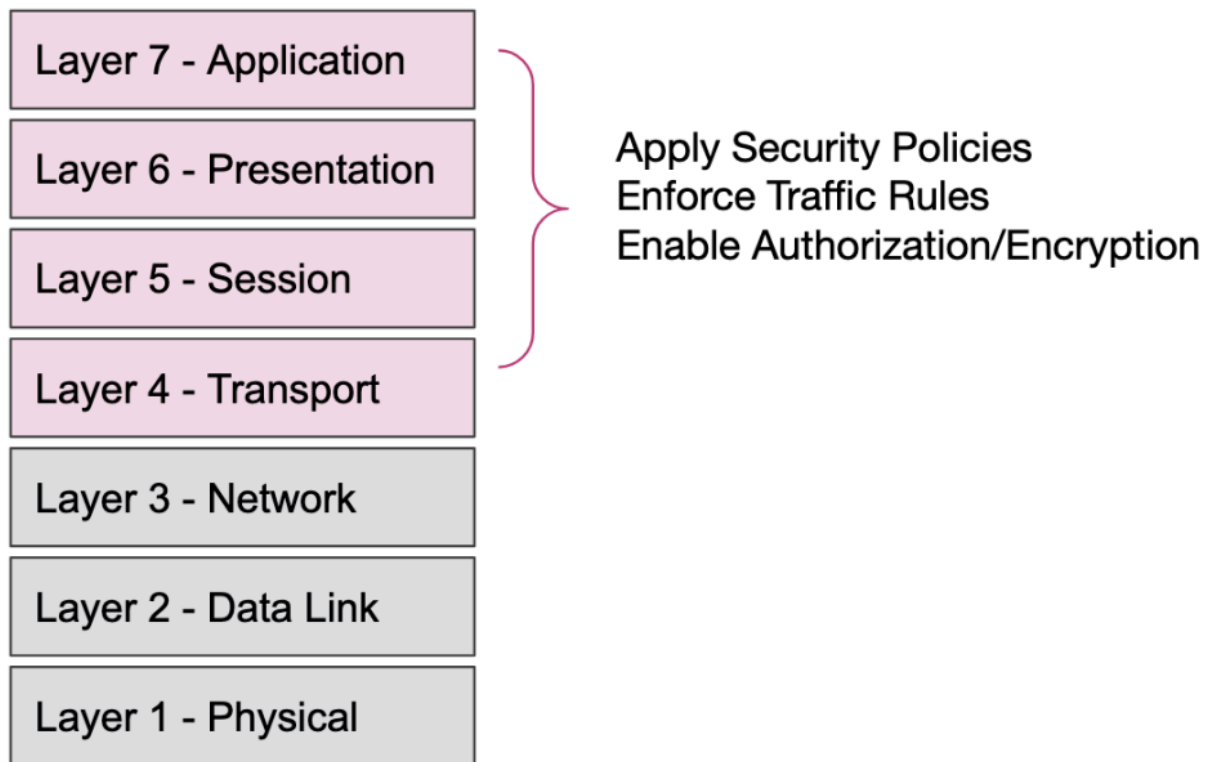
Networking still needs to be based on service identity because that's how orchestrators track things, but the shift away from static, IP-based networking toward a service-based networking solution that these service discovery features provided was perhaps the most impactful change to networking, making application identity the basis for **networking policies**.

The Impact on Security

Networking's transition to a service-identity-based networking requirement also has cascading effects on other workflows. The first, and arguably the most important, is on security. While service discovery may solve for tracking changes more dynamically, it doesn't help you apply consistent security policies to those applications.

As I mentioned earlier, enforcing security and access to sensitive data is a core networking requirement. The reason we employ firewalls and load balancers is to help ensure that only the correct services are granted access to the network. And that approach works fine if you're relying on IP addresses to track what's running on the network. But what do we do now that we're using service identity instead?

create enforcement mechanisms around those identities. Unfortunately, traditional **network security** policies don't really support that pattern. That approach forces development teams to rely on ticket-based systems and creates bottlenecks and frustration for both operators and developers. Fortunately, there are two ways to address this challenge: eliminating manual tasks for operators and moving security policies from Layer 3 to Layer 7.

[Zoom](#)

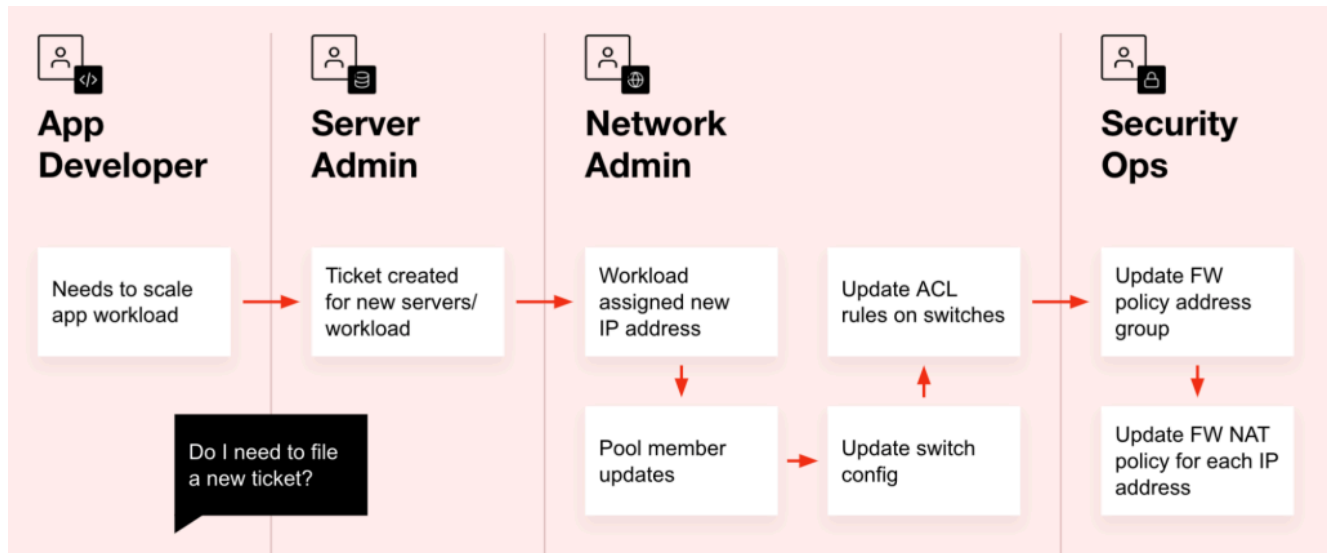
Eliminating the Ticket-Based Bottlenecks

Let's start with eliminating manual tasks. To understand how this works, it's helpful to look at another modernized workflow: infrastructure provisioning. Tools like HashiCorp Terraform and CloudFormation were built to eliminate the process of manually configuring infrastructure. When we talk about security automation, it can be tempting to focus on the future state of a solution, but that implies companies are ready to completely give up on their existing environments and investments.

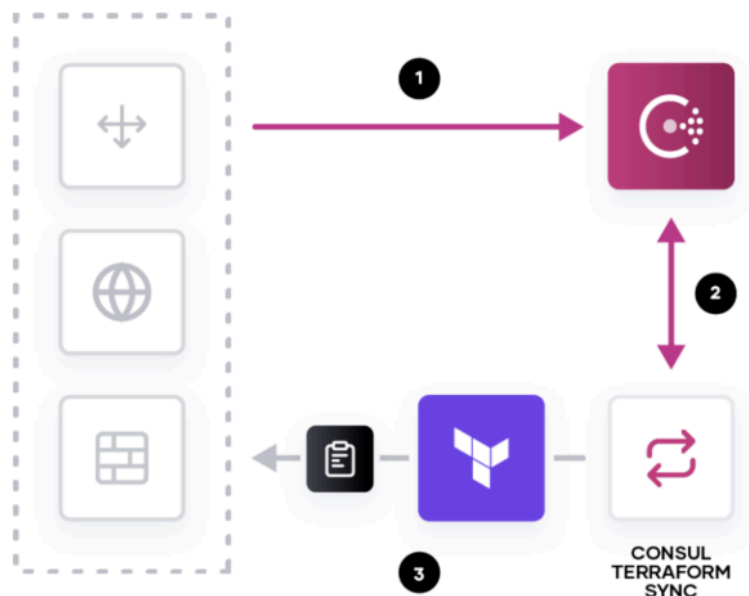
That may not always be such a good idea.

In a service-based networking environment, it's all about building **security in depth**. Firewall solutions still play an important role in modern data centers, but that doesn't mean updates and policy enforcement can't be automated. It's just that

changes as events that have subsequent tasks that need to be performed. You may have heard of this concept being referred to as configuration management or Day 2 management, and it's what tools like Consul-Terraform-Sync are designed to achieve.



Zoom



Moving Security Policies from Layer 3 to Layer 7

However, as users push services into newer, cloud-based environments, they may look to move away from some of their existing infrastructure and explore other security approaches, namely moving security policies from Layer 3 to Layer 7. Essentially, this means:

- Enforcing traffic management policies based on service identity
- Requiring mutual authentication

If these approaches sound familiar, it's because **that's exactly what a service mesh does**. Which may explain why we've seen an explosion of new service mesh offerings. Service mesh is also becoming a common pattern for building **zero trust security** at the networking layer, meaning traffic inside your network is untrusted by default and must verify each connection to a service.

For more on service mesh, see these articles on The New Stack:

- [Service Mesh Is Just the Tip of the Iceberg](#)
- [For My Next Trick, I'll Make a Service Mesh... Disappear!](#)

Enabling Automation

The next core workflow for a service-based networking environment is enabling automation. One of the prevailing themes in modern infrastructure is the idea of being able to create templates for building repeatable environments in the cloud, also known as **Infrastructure-as-Code**. However, this concept has extended beyond the scope of just cloud compute resources. You can do just about anything with a Terraform provider these days, from **ordering a pizza** to **creating a Spotify playlist**.

But is this really automation? Templatizing environments and making them repeatable is great, but someone is ultimately on the other end, still triggering the changes. What about ongoing management? What about that Day 2 story mentioned earlier? Because applications are continually added or removed depending on the needs of the organization, modern networking is all about the ongoing management of a given environment. How do we apply that ongoing automation in our service-based networking story?

We have a couple of options, which align to the solutions outlined in the security section above. First, you can apply automation principles to key networking hardware used to regulate connections between services. In a more traditional environment, you might rely on something like an application delivery controller (ADC) to manage the rules for how services interact. However, these systems rely on IP addresses and are often manually configured. You could use a provisioning tool, but that solution just changes how you apply changes; it doesn't remove the actual interaction piece.

Head, the goal should be trying to use a system that dynamically changes as the services change. After all, you're tracking the addition or removal of applications,

as network infrastructure automation (NIA) and it's something that Consul and Terraform, which also is open source, can achieve working together.

Additionally, as organizations shift towards newer environments, they often look to apply traffic management policies or automate connectivity between services based on their identity. Ideally, you'd set up rules that say which services are allowed to communicate with one another. Regardless of how many instances of each service are deployed, they would follow the same rules defined by the service identity. Sound familiar? It should, because once again it's something that a service mesh can assist with.

At this point, you might be wondering why I haven't just claimed that a service mesh solves everything. That's because a service mesh is just a part of a broader solution. The shift to service-based networking is at the heart of how a service mesh operates, but a service mesh is not applicable to every environment. To outline a true solution to the workflow, we have to explore all the environments and application types a service-based network applies to, not just microservices and containers.

Controlling Network Access

Now that we've covered the need to discover services in a dynamic fashion, applying security policies based on identity and events, and incorporating automation into our networking solutions, what's left? None of these things really matter if end users and clients can't actually access the network or the services running on it.

Thinking back to the way access used to be regulated with ADCs and firewalls, we quickly run into the same scaling and bottleneck issues identified earlier. Additionally, once you transition networking policies to a service-based **networking solution**, maintaining traditional networking technologies can actually hinder the ability to achieve consistency. Ideally, you want to apply the same service-based traffic management solution for managing north-south (ingress/egress) traffic as you do for east-west (service-to-service) traffic.

Kubernetes users reading this might think that there is already a solution to this challenge: ingress controllers. The general value proposition is that ingress controllers allow access to a Kubernetes cluster with the ability to apply certain traffic-management capabilities and security protocols. The benefit is that these

always as dynamic as users might desire. Oftentimes, they require longer rollout cycles to increase the number of replicas in a given cluster.

To address this dynamism challenge, the Kubernetes community introduced the **Gateway API specification** to enhance what was currently supported by ingress solutions. The Gateway API spec allows for more dynamic management of gateway services, routes, and is more closely aligned to a service mesh. That's why we built the **Consul API Gateway** as an implementation of the Kubernetes Gateway API spec. This enables users to create a consistent control plane for all traffic management and access to the service mesh.

If you're unfamiliar with the Kubernetes Gateway API specification, it lets users deploy a dedicated gateway pod into a cluster, then dynamically apply routes via a custom resource definition. Using this model, Kubernetes users can deploy multiple versions of a gateway as needed within the cluster, and apply specific routing policies or listener protocols to each gateway instance. When combined with a service mesh, users can control traffic policies for both north-south and east-west traffic using a single solution and all based on service identities.

And, of course, the typical enterprise environment encompasses quite a bit more than just Kubernetes. Beyond Kubernetes, we need to apply these same policies and practices to all environments. Containers can be run by **multiple orchestrators** and some applications **will never be containerized**. We still need to have that central control plane for managing access to the network based on service identities. Doing this creates consistency across both cloud and non-cloud environments.

Conclusion

As we've seen, when addressing the modern networking challenge, users need to solve for four key workflows: discovering services, securing networks, automating networking tasks and controlling access. These four workflows comprise the core tenets of service-based networking.

It's important to remember that the industry shift to service-based networking evolved over a number of years. That's why service networking solutions continue to implement new functionality. Even as networking continues to evolve, however, the core goal remains: moving away from IP-based networking to service-based networking. Embracing this change allows networks to match the velocity of

TNS



Peter is a senior product marketing manager for Consul at HashiCorp and based in San Francisco. If he's not studying the best way to discover and manage microservices or talking about cloud-based networking, you'll likely find him discovering real clouds...

Read more from Peter McCarron →

Hashicorp is a sponsor of The New Stack.

TNS owner Insight Partners is an investor in: Pragma, Docker.

SHARE THIS STORY



TRENDING STORIES

1. HTTP/3 Is Now a Standard: Why Use It and How to Get Started
2. Linux: Display and Manage IP Address Settings
3. How To Read a Traceroute for Network Troubleshooting
4. From system of record to system of control: How NetBox Labs is making network engineers "masters of intent."
5. How Bolt Scaled Remote Work with Tailscale's Zero Trust Mesh VPN

INSIGHTS FROM OUR SPONSOR



[Learn More →](#)

Deploy Boundary on Kubernetes with official Helm charts

25 June 2026

Boundary 1.0 releases RDP session recording and improved management

25 June 2026

Scaling without friction: Aliases at project scope in Boundary

25 June 2026

HCP Vault Dedicated introduces cluster disaster recovery (public preview)

25 June 2026

Advancing AI agent security in Vault

24 June 2026

Introducing tfctl: The CLI for HCP Terraform and TFE

16 June 2026

TNS DAILY NEWSLETTER

**Receive a free roundup of the
most recent TNS articles in
your inbox each day.**

svo1975pnv1982@gmail.com

ARCHITECTURE

Cloud Native Ecosystem
Containers
Databases
Edge Computing
Infrastructure as Code
Linux
Microservices
Open Source
Networking
Storage

ENGINEERING

AI
AI Engineering
API Management
Backend development
Data
Frontend Development
Large Language Models
Security
Software Development
WebAssembly

OPERATIONS

AI Operations
CI/CD
Cloud Services
DevOps
Kubernetes
Observability
Operations
Platform Engineering

CHANNELS

Podcasts
Ebooks
Events
Webinars
Newsletter
TNS RSS Feeds

THE NEW STACK

About / Contact
Sponsors
Advertise With Us
Contributions



Community created roadmaps, articles, resources and journeys for developers to help you choose your path and grow in your career.

FOLLOW TNS



© The New Stack 2026

[Disclosures](#)

[Terms of Use](#)

[Advertising Terms & Conditions](#)

[Privacy Policy](#)

[Cookie Policy](#)